

AI – BASED CYBER DEFENSE SYSTEM

GenAI-Powered Employee & Banking Transaction Protection System

Learning Block 1

Submitted by

THUMMALAPALLY AKSHITHA

TEEGALA KRISHNA REDDY ENGINEERING COLLEGE

Department of CSE

Academic Year: Final Year – [2026–2027]

Guided by: Raghavendra Rao

[GitHub Link: https://github.com/takshitha123/AI-Based-Cyber-Defense-System](https://github.com/takshitha123/AI-Based-Cyber-Defense-System)

TABLE OF CONTENTS

S.No	Topic / Chapter Name	Page No.
1	Introduction (Overview, GenAI Role, Output Modalities, Key Features)	3
2	Problem Statement (Real-World Inefficiencies & GenAI Advantages)	4
3	Objectives (Technical Milestones & Evaluation Criteria)	4
4	Project Scope (Target Users, Inputs & Operational Boundaries)	5
5	Proposed System / Methodology (Evidence-First GenAI Pipeline & Logic)	5
6	System Architecture / Workflow (GenAI Security Architecture)	7
7	Implementation (Tech Stack, Risk Engine & GenAI Integration)	8
7.1	Program Code – GenAI Integration	9
7.2	Program Code – Risk Engine	9
7.3	Program Code – Evidence Collection & Investigation	10
7.4	Program Code – Decision Enforcement	11
7.5	Program Code – Controlled Attack Simulation	12
7.6	Program Code – Protected Bank Transaction	13
8	User Interface / Application Screenshots	14
8.1	Security Operations Center (SOC) Dashboard	14
8.2	Synthetic Bank Accounts	14
8.3	Employee Banking Portal & Normal Transaction	15
8.4	Controlled Attack Lab	16
8.5	Controlled Attack Lab – Blocked Simulation Result	16
8.6	Incident Investigation – GenAI Security Report	16
8.7	Incident Investigation – Security Evidence & Telemetry	17
8.8	Incident Investigation – Audit Timeline	17
8.9	Incident Investigations Dashboard	18
9	Challenges and Limitations (Engineering Solutions & Scope Limits)	18
10	Conclusion (Key Findings, GenAI Utility & Learning Outcomes)	19
11	Future Scope (Deployment, Scaling & Advanced GenAI)	19
12	References (GenAI, API & Framework Documentation)	19

1. Introduction

CyberGuard AI is a functional GenAI-powered cybersecurity application designed to protect a synthetic employee banking environment from suspicious authentication, session, behavioural, and transaction activity. Instead of presenting a static cybersecurity dashboard, the system creates a working flow in which real requests enter a CyberGuard Gateway, evidence is collected from the live demonstration environment, risk is calculated, a GenAI security analyst interprets the supplied evidence, and the final decision is enforced before the synthetic bank commits the transaction.

The application addresses a practical security problem: an employee account can have a valid password and a valid session while the surrounding context becomes abnormal. A security system therefore needs to evaluate more than credentials alone. CyberGuard combines device trust, failed authentication activity, transaction history, beneficiary context, request frequency, session/device consistency, and sensitive-operation context.

The specific role of Generative AI is the investigation and explanation layer. The deterministic detector first produces structured, evidence-backed indicators and a 0–100 risk score. When GenAI is configured, CyberGuard sends the observed evidence and deterministic risk context to a Gemini-compatible Generative AI analyst. The model is instructed to use only supplied evidence, return structured JSON, separate observation from interpretation, and report uncertainty instead of inventing facts. The generated output contains classification, confidence, severity, recommended action, summary, reasoning, evidence references, investigation steps, and uncertainty.

The application can generate a practical security investigation report rather than only a numerical score. When the GenAI service is unavailable, a deterministic evidence-backed fallback analyst is used and the fallback status is recorded. This makes the application operationally safer and demonstrates that the enforcement layer does not depend on an unverified AI response.

Main user-facing features

- Functional synthetic banking portal with user-created accounts, demo balances, login, beneficiaries, transfers, and transaction history.
- CyberGuard Gateway that intercepts banking requests and separates observation, analysis, and enforcement from the protected bank server.
- Controlled Attack Lab that produces real telemetry inside the synthetic environment without malware or attacks against third-party systems.
- Evidence-grounded GenAI investigation that explains observed indicators and links important claims to stored event evidence.
- Real-time incident, decision, and audit views showing why a request was allowed, isolated, or blocked.

The current project deliberately uses synthetic money and a controlled environment. It does not use real money, real credentials, malware, or external targets.

2. Problem Statement

Banking systems receive continuous login, session, API, and transaction activity. Traditional security controls can identify known patterns, but a difficult case occurs when a legitimate employee account is used from an unusual device or after abnormal authentication activity. A single rule may not provide enough context to distinguish a genuine employee from a compromised session.

The project therefore addresses the following problem: how can a banking security layer detect abnormal employee activity, gather concrete evidence, use Generative AI to investigate that evidence, and enforce a protective decision before a suspicious transaction reaches the protected banking service?

Why GenAI is useful

- Rule-based detection is useful for identifying measurable signals such as failed logins or unusual transaction amounts, but it does not naturally produce a contextual investigation narrative.
- GenAI can combine the structured evidence and deterministic risk indicators into a readable security assessment containing classification, confidence, recommended action, evidence references, and investigation steps.
- The prompt explicitly constrains the model to supplied evidence. This reduces unsupported conclusions and makes the generated explanation auditable.
- The system records the generated investigation alongside the incident, request, telemetry, and audit timeline, making the AI output part of an evidence trail rather than an isolated chatbot response.

Expected outcome

When the system is used, normal banking activity can continue through an ALLOW path, while suspicious activity can be investigated and blocked or isolated before the synthetic bank commits the transaction. The user can inspect the underlying telemetry and the AI-generated investigation instead of relying on an unexplained security score.

3. Objectives

1. Develop a functional and deployable GenAI-based cybersecurity application around a synthetic banking use case.
2. Integrate a Gemini-compatible Generative AI analyst through environment-based configuration without exposing credentials in the frontend.
3. Collect real synthetic security telemetry and construct structured evidence for every investigation.
4. Calculate an explainable 0–100 deterministic risk score from measurable security indicators.
5. Use GenAI to produce evidence-grounded classification, confidence, recommended action, investigation steps, and uncertainty.
6. Enforce ALLOW, ISOLATE, or BLOCK before the protected bank commits a suspicious transaction.
7. Maintain incidents, AI investigations, security events, decisions, and audit logs in persistent storage.
8. Provide a practical live demonstration in which a normal transfer succeeds and a controlled suspicious transaction is prevented.

4. Project Scope

In scope

- Synthetic employee banking accounts and demo balances.
- Employee authentication, device identifiers, sessions, beneficiaries, transfers, and transaction history.

- CyberGuard request interception and telemetry collection.
- Deterministic risk assessment using device, authentication, transaction, session, and behavioural indicators.
- GenAI investigation using structured observed evidence.
- Incident generation, evidence references, audit trail, and decision enforcement.
- Safe controlled attack scenarios: credential compromise, account takeover, transaction fraud, and session anomaly.
- Deployment-ready configuration through environment variables.

Target users

- Bank security/SOC demonstrators and analysts who need an understandable investigation trail.
- Developers and students demonstrating a functional GenAI cybersecurity architecture.
- Security operators who need a human-verification path for uncertain events.

Inputs and outputs

Input	System processing	Output
Login credentials + device ID	Authentication + session telemetry	Session / AUTH FAILURE evidence
Transfer request	Context collection + risk assessment	Risk score + decision
Observed events and transaction context	Prompt construction + GenAI analysis	Classification, confidence, reasoning, evidence, steps, uncertainty
Controlled attack scenario	Real synthetic HTTP activity	Incident + audit timeline + enforcement result

Boundaries

The current version is a synthetic banking/security demonstration. It does not process real money, attack real banking systems, deploy malware, or claim production banking certification. SQLite is suitable for the demonstration; production-scale deployment would require a persistent production database and stronger operational controls.

5. Proposed System / Methodology

CyberGuard follows an evidence-first methodology. The security layer does not begin by asking a language model to guess whether an event is malicious. Instead, measurable telemetry is collected first, deterministic risk indicators are calculated, and the GenAI analyst receives the resulting evidence as structured input.

End-to-end process

9. User interaction: an employee logs into the synthetic bank and performs a legitimate or controlled test operation.
10. Request interception: the CyberGuard Gateway receives the request and maintains the security context.
11. Evidence collection: authentication outcomes, device context, session information, request activity, transaction history, beneficiary context, and related events are assembled.
12. Deterministic analysis: the risk engine evaluates indicators such as UNKNOWN_DEVICE, AUTH_ANOMALY, NEW_BENEFICIARY, AMOUNT_DEVIATION, RAPID_ACTIVITY, SESSION_DEVICE_MISMATCH, and SENSITIVE_OPERATION.
13. Risk scoring: indicator weights are summed and capped at 100 to produce a LOW, MEDIUM, or HIGH risk level.
14. GenAI investigation: when configured, the Gemini-compatible analyst receives the observed evidence and deterministic risk context and returns structured JSON.
15. Evidence verification: the application keeps the observed telemetry and AI evidence references together; the prompt requires the model not to invent unsupported facts.

16. Decision enforcement: CyberGuard applies ALLOW, ISOLATE, or BLOCK before the protected bank commits the operation.
17. Persistence: incidents, events, AI investigation output, decisions, and audit information are stored for later inspection.

GenAI prompt strategy

The GenAI input contains an observed_evidence object, deterministic risk score and level, detected indicators, and a required structured output schema. The system instruction identifies the model as a security analyst and requires JSON output using only supplied evidence. The prompt explicitly says to separate observed evidence from interpretation, state uncertainty when evidence is insufficient, and not provide hidden chain-of-thought.

AI output

- Classification
- Confidence
- Severity
- Recommended action
- Summary
- Evidence references
- Investigation steps
- Uncertainty

This makes GenAI a functional part of the security workflow: it transforms structured security evidence into an auditable investigation artifact that is displayed in the incident view and stored with the incident.

6. System Architecture / Workflow

CyberGuard AI — GenAI Security Architecture

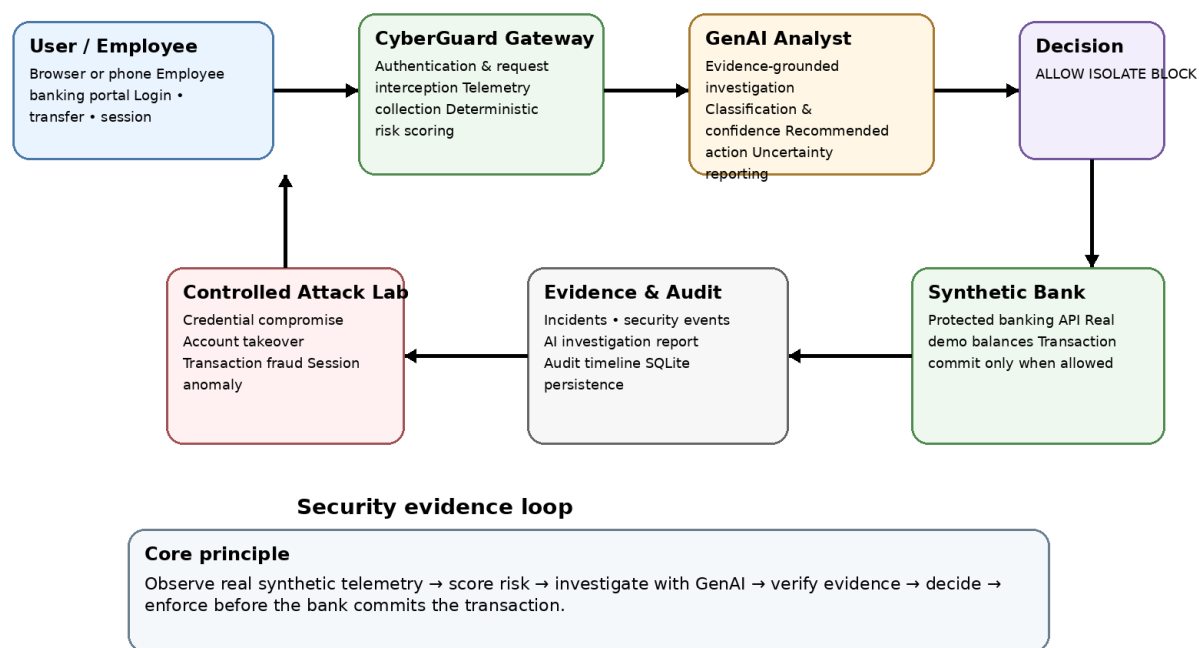


Figure 2. CyberGuard AI architecture and evidence-to-decision workflow.

Architecture components

Component	Responsibility
Synthetic Bank Server	Maintains demo accounts, balances, beneficiaries, sessions and transactions; commits a transaction only when permitted through the protected path.
CyberGuard Gateway	Public application entry point; intercepts requests, authenticates context, records telemetry, invokes analysis, and enforces the decision.
Detection / Risk Engine	Converts observed security context into deterministic indicators and a 0–100 risk score.
GenAI Analyst	Interprets supplied evidence and returns structured security investigation output.
Incident & Audit Store	Persists incidents, events, AI investigations, decisions and audit records.
Controlled Attack Lab	Generates safe, internal test activity that exercises the detection and enforcement path.
Frontend / SOC Console	Presents accounts, banking workflow, incidents, attack scenarios, evidence, AI investigation, and audit information.

Security enforcement principle

The protected bank server is not the place where the AI makes its final decision. CyberGuard makes the security decision first and only forwards an allowed request to the bank. This separation creates a clear control point: suspicious activity can be stopped before the synthetic bank changes its balance.

7. Implementation

Technology stack

Layer	Technology / implementation
Language	Python 3.13.x
Backend	FastAPI + Uvicorn
Frontend	HTML, CSS and JavaScript single-page interface
Persistence	SQLite for the functional demonstration
HTTP integration	HTTPX for service-to-service and GenAI API communication
Configuration	Environment variables via python-dotenv
GenAI	Gemini-compatible Generative AI API; default model configured as gemini-2.5-flash
Security logic	Deterministic evidence-based risk engine + policy decision engine
Testing	Pytest-based automated tests

GenAI integration

The implemented GenAI integration reads `GENAI_API_KEY`, `GENAI_PROVIDER`, `GENAI_MODEL`, and `GENAI_BASE_URL` from the environment. The default provider is Gemini and the default model configuration is `gemini-2.5-flash`. The backend sends the structured evidence to the configured Generative AI endpoint using an HTTP request and requests JSON output.

If the API key is missing or the model call fails, the application switches to an evidence-backed deterministic fallback analyst and records that fallback status. This prevents a failed AI dependency from silently producing an untraceable or invented security conclusion.

Risk scoring implementation

Indicator	Weight	Meaning
UNKNOWN_DEVICE	22	Device is not trusted for the user.
AUTH_ANOMALY	20	At least three recent failed authentication attempts.

NEW_SESSION	10	A new session was created.
NEW_BENEFICIARY	16	Transfer targets an unused beneficiary.
AMOUNT_DEVIATION	18	Amount is materially outside observed range.
RAPID_ACTIVITY	14	At least eight recent security-relevant requests.
SESSION_DEVICE_MISMATCH	24	Session is used from unexpected device context.
SENSITIVE_OPERATION	8	Financial operation requires elevated trust.

The final score is capped at 100. The current risk engine classifies scores below 30 as LOW, 30–69 as MEDIUM, and 70 or above as HIGH. The decision layer then applies the security policy to allow, isolate, or block the request.

Controlled attack evidence

The Attack Lab does not insert fake AUTH_FAILURE rows directly into the database. For credential-compromise/account-takeover simulations, it makes six real HTTP authentication requests to the synthetic bank's protected login endpoint using controlled invalid credentials. The synthetic bank returns HTTP 401 for each rejected request, and CyberGuard records each result as telemetry with an event ID, timestamp, device ID, session ID, endpoint, request ID, HTTP status, and attempt number. The resulting incident can therefore trace an aggregate statement such as 'six failed authentication attempts' back to concrete event IDs.

7.1 Program Code – GenAI Integration

Core GenAI integration: structured evidence is sent to the configured Gemini-compatible model and a JSON investigation is returned. The application falls back to an evidence-backed analyst when the API is unavailable.

```
async def analyze(e, score, level, inds):
    key = env("GENAI_API_KEY")
    base = env("GENAI_BASE_URL").rstrip("/")
    model = env("GENAI_MODEL", "gemini-2.5-flash")

    if not key:
        return "fallback", fallback(e, score, level, inds), "GENAI_API_KEY is not configured"

    prompt = {
        "observed_evidence": e,
        "deterministic_risk": {
            "score": score,
            "level": level,
            "indicators": inds
        },
        "required_output": [
            "classification", "confidence", "severity",
            "recommended_action", "summary", "reasoning",
            "evidence", "investigation_steps", "uncertainty"
        ],
        "rule": (
            "Use only supplied evidence; never invent facts. "
            "Separate observed evidence from interpretation. "
            "If evidence is insufficient, say uncertain."
        )
    }

    url = f"{base}/models/{model}:generateContent?key={key}"
    body = {
        "system_instruction": {
            "parts": [{"text":
                "You are a security analyst. Return JSON only. "
                "Use only observed evidence."}]
        },
        "contents": [{"parts": [{"text": json.dumps(prompt)}]},
        "generationConfig": {"responseMimeType": "application/json"}
    }

    async with httpx.AsyncClient(timeout=20) as client:
        response = await client.post(url, json=body)
        response.raise_for_status()
        data = response.json()
        text = data["candidates"][0]["content"]["parts"][0]["text"]
        return "genai", json.loads(text), None
```

7.2 Program Code – Risk Engine

The deterministic risk engine produces explainable indicators and a 0–100 score before the GenAI investigation.

```
def assess(e):
    inds = []

    def add(key, weight, evidence):
        inds.append({
            "id": key,
            "weight": weight,
            "evidence": evidence
        })

    if e.get("unknown_device"):
        add("UNKNOWN_DEVICE", 22,
            "Device is not trusted for this user.")

    if e.get("failed_auths", 0) >= 3:
        add("AUTH_ANOMALY", 20,
            f"{e['failed_auths']} failed authentication attempts "
            "were observed recently.")
```

```

if e.get("new_session"):
    add("NEW_SESSION", 10, "A new session was created.")

if e.get("new_beneficiary"):
    add("NEW_BENEFICIARY", 16,
        "The transfer targets an unused beneficiary.")

if e.get("amount_deviation"):
    add("AMOUNT_DEVIATION", 18,
        "The amount is materially outside the observed range.")

if e.get("rapid_activity"):
    add("RAPID_ACTIVITY", 14,
        f"{e.get('recent_requests', 0)} recent security requests.")

if e.get("session_device_mismatch"):
    add("SESSION_DEVICE_MISMATCH", 24,
        "The active session uses an unexpected device.")

if e.get("sensitive_operation"):
    add("SENSITIVE_OPERATION", 8,
        "The request is a sensitive financial operation.")

score = min(100, sum(i["weight"] for i in inds))
level = "LOW" if score < 30 else "MEDIUM" if score < 70 else "HIGH"
return score, level, inds

```

7.3 Program Code – Evidence Collection & Investigation

The evidence layer collects recent authentication failures and device trust, while the investigation function persists the resulting AI report with the incident.

```

def history(user_id, device_id):
    recent = one(
        "SELECT COUNT(*) c FROM events "
        "WHERE user_id=? AND ts>=datetime('now','-10 minutes')",
        (user_id,)
    )["c"]

    failed = one(
        "SELECT COUNT(*) c FROM events "
        "WHERE user_id=? AND type='AUTH_FAILURE' "
        "AND ts>=datetime('now','-10 minutes')",
        (user_id,)
    )["c"]

    failed_events = rows(
        "SELECT id,ts,device_id,session_id,type,data "
        "FROM events WHERE user_id=? AND type='AUTH_FAILURE' "
        "AND ts>=datetime('now','-10 minutes') ORDER BY ts ASC",
        (user_id,)
    )

    trusted = one(
        "SELECT trusted FROM devices WHERE id=? AND user_id=?",
        (device_id, user_id)
    )

    return {
        "recent_requests": recent,
        "failed_auths": failed,
        "failed_auth_events": failed_events,
        "device_trusted": bool(trusted and trusted["trusted"])
    }

async def investigate(request_id, evidence, score, level, indicators):
    status, result, error = await analyze(
        evidence, score, level, indicators
    )

    incident_id = "INC-" + uuid.uuid4().hex[:8].upper()

    execute(
        "INSERT INTO incidents VALUES(?,?,?,?,?,?,?,?,?,?)",

```

```

        (
            incident_id, request_id, now(), "ACTIVE",
            result.get("classification"), score,
            result.get("confidence"), result.get("summary"),
            result.get("recommended_action"),
            j(result.get("evidence", [])), j(result)
        )
    )
    return incident_id, status, result, error

```

7.4 Program Code – Decision Enforcement

The decision layer is the security boundary: only ALLOW is forwarded to the protected bank. BLOCK and ISOLATE are recorded without committing the transfer.

```

score, level, indicators = assess(evidence)

if score >= 30:
    incident_id, ai_status, ai_report, error = await investigate(
        request_id, evidence, score, level, indicators
    )
    recommendation = ai_report.get(
        "recommended_action", "human_verification"
    )
else:
    incident_id = None
    ai_status = "not_required"
    ai_report = None
    recommendation = "allow"

decision = (
    "BLOCK" if score >= 80 else
    "ISOLATE" if score >= 45
    or recommendation in ("isolate", "human_verification")
    else "ALLOW"
)

if decision == "ALLOW":
    bank_result = await bank(
        "POST",
        "/internal/transfer",
        json={
            "from_account": source_account["id"],
            "to_account": transfer.to_account,
            "amount": transfer.amount,
            "request_id": request_id,
            "reason": "CyberGuard allowed"
        }
    )
    forwarded = True
else:
    forwarded = False
    event(
        user_id, device_id, session_id,
        "TRANSACTION_BLOCKED",
        {"request_id": request_id,
         "amount": transfer.amount,
         "decision": decision}
    )

```

7.5 Program Code – Controlled Attack Simulation

The Attack Lab creates six real rejected login requests against the synthetic bank. Each HTTP 401 response becomes concrete CyberGuard telemetry.

```

for attempt_no in range(1, 7):
    auth_request_id = "REQ-" + uuid.uuid4().hex[:10].upper()

    try:
        await bank(
            "POST",
            "/internal/login",
            json={
                "user_id": attack.user_id,

```

```

        "password":
            f"controlled-invalid-{uuid.uuid4().hex[:12]}"
    }
)

event(
    attack.user_id, device_id, attack_session,
    "AUTH_UNEXPECTED_SUCCESS",
    {
        "source": "controlled_attack_lab",
        "endpoint": "/internal/login",
        "attempt_number": attempt_no,
        "request_id": auth_request_id
    }
)

except HTTPException as ex:
    if ex.status_code != 401:
        raise

    event(
        attack.user_id, device_id, attack_session,
        "AUTH_FAILURE",
        {
            "source": "controlled_attack_lab",
            "endpoint": "/internal/login",
            "attempt_number": attempt_no,
            "request_id": auth_request_id,
            "http_status": 401,
            "result": "HTTP_401_INVALID_CREDENTIALS"
        }
    )
)

```

7.6 Program Code – Protected Bank Transaction

The protected synthetic bank changes balances only after CyberGuard forwards an allowed request. This is what makes the blocked-transaction demonstration verifiable.

```

def transfer(x: TransferIn,
            x_cyberguard_secret: str = Header(default="")):
    auth(x_cyberguard_secret)

    c = db.conn()
    c.execute("BEGIN IMMEDIATE")

    src = c.execute(
        "SELECT * FROM accounts WHERE id=?",
        (x.from_account,)
    ).fetchone()

    dst = c.execute(
        "SELECT * FROM accounts WHERE id=?",
        (x.to_account,)
    ).fetchone()

    if not src or not dst:
        c.rollback()
        raise HTTPException(404, "Account not found")

    if src["balance"] < x.amount:
        c.rollback()
        raise HTTPException(400, "Insufficient funds")

    c.execute(
        "UPDATE accounts SET balance=balance-? WHERE id=?",
        (x.amount, x.from_account)
    )
    c.execute(
        "UPDATE accounts SET balance=balance+? WHERE id=?",
        (x.amount, x.to_account)
    )

    transaction_id = "TXN-" + uuid.uuid4().hex[:10].upper()

```

```

c.execute(
    "INSERT INTO transactions VALUES(?,?,?,?,?,?,?)",
    (
        transaction_id, db.now(), x.from_account,
        x.to_account, x.amount, "SUCCESS",
        x.reason, x.request_id
    )
)

c.commit()
c.close()

return {
    "transaction_id": transaction_id,
    "status": "SUCCESS",
    "amount": x.amount
}

```

Code note: These are the principal executable sections relevant to the Learning Block 1 evaluation. The complete source remains organized in the gateway, bank_server, frontend and tests directories.

8. User Interface / Application Screenshots

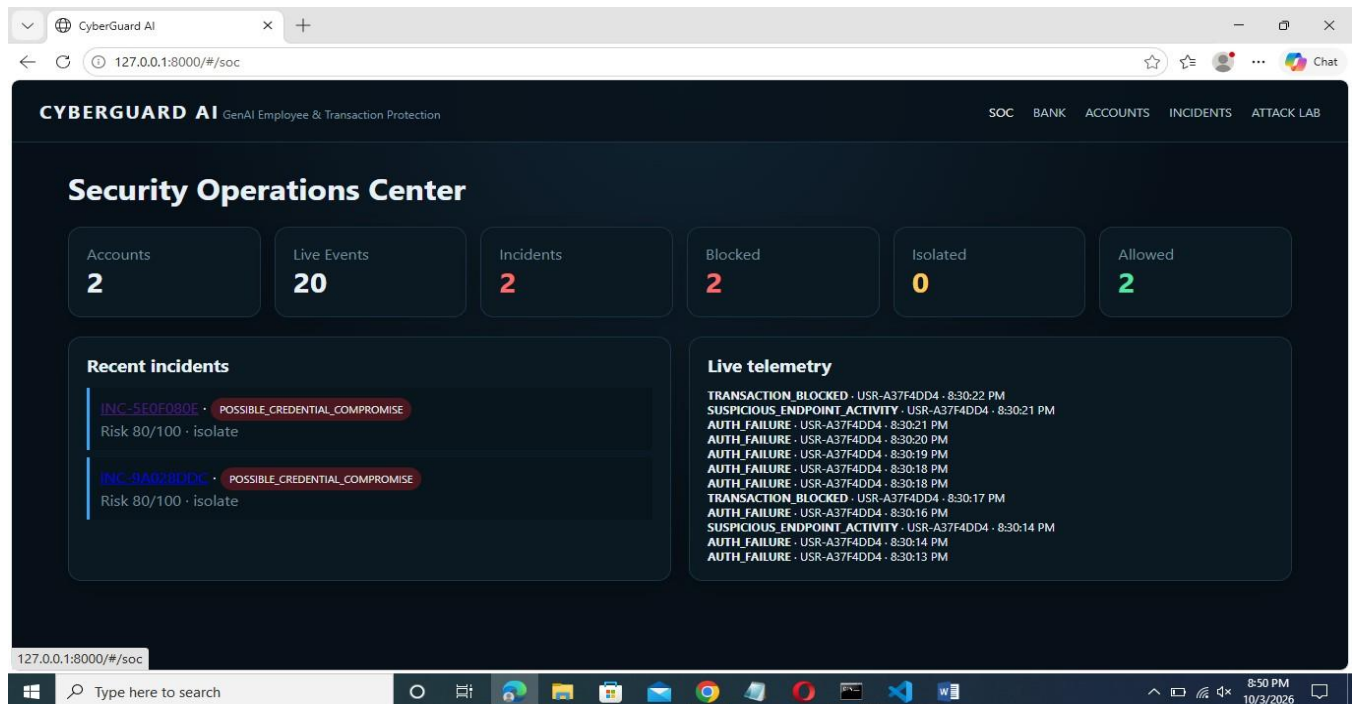


Fig : 8.1 Security Operations Center (SOC) Dashboard

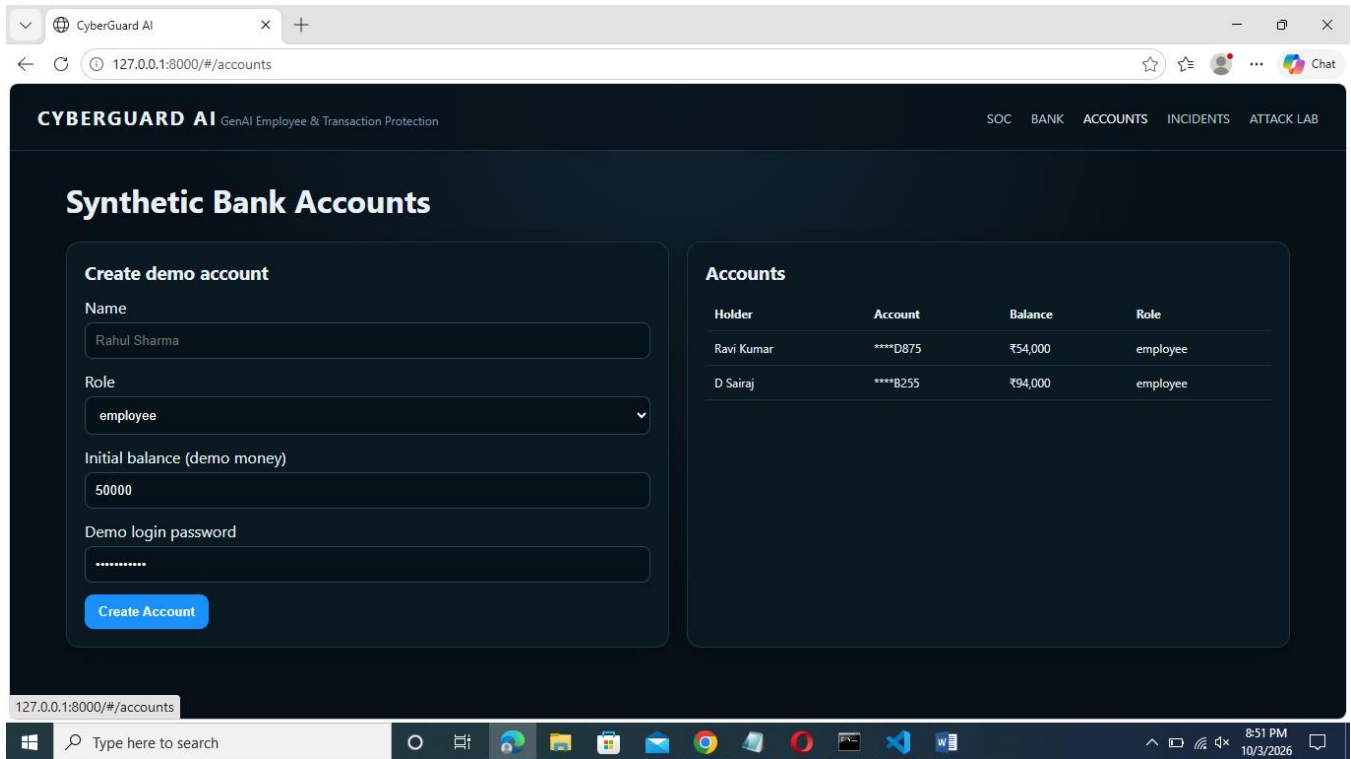


Fig: 8.2 Synthetic Bank Accounts

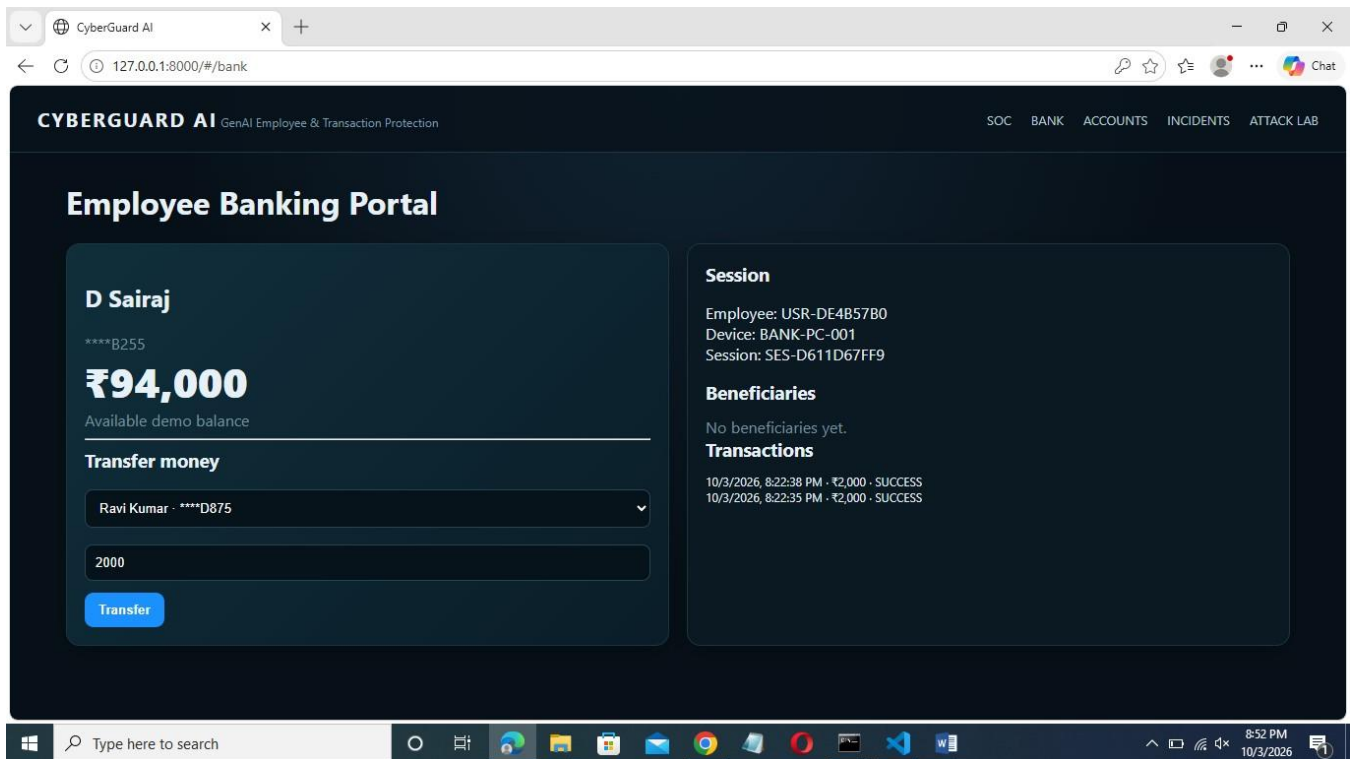


Fig : 8.3 Employee Banking Portal & Normal Transaction

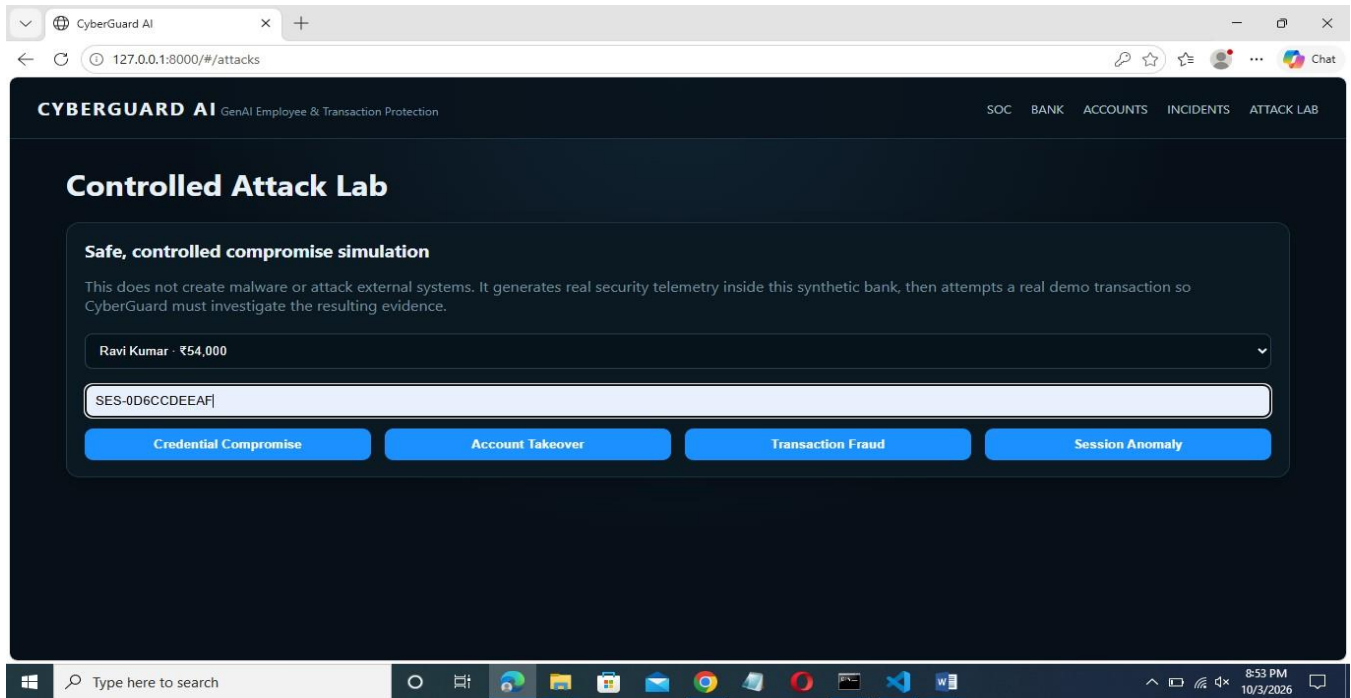


Fig: 8.4 Controlled Attack Lab

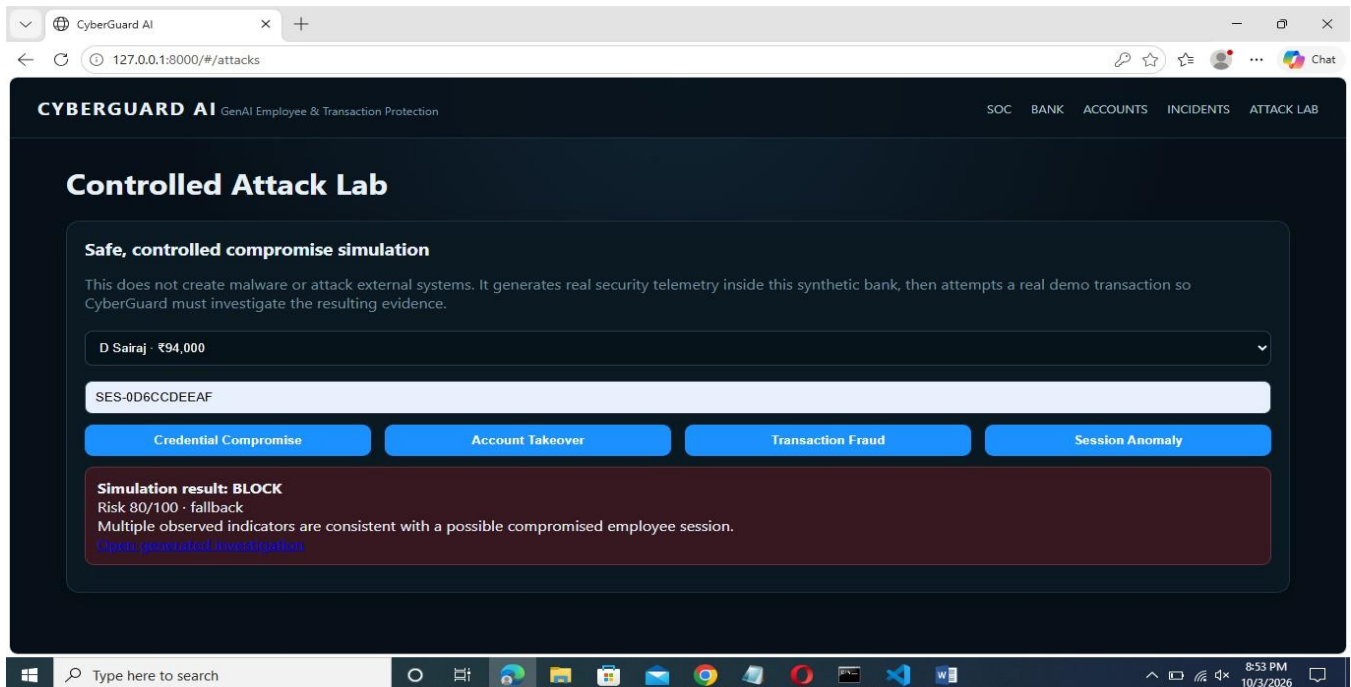


Fig: 8.5 Controlled Attack Lab – Blocked Simulation Result

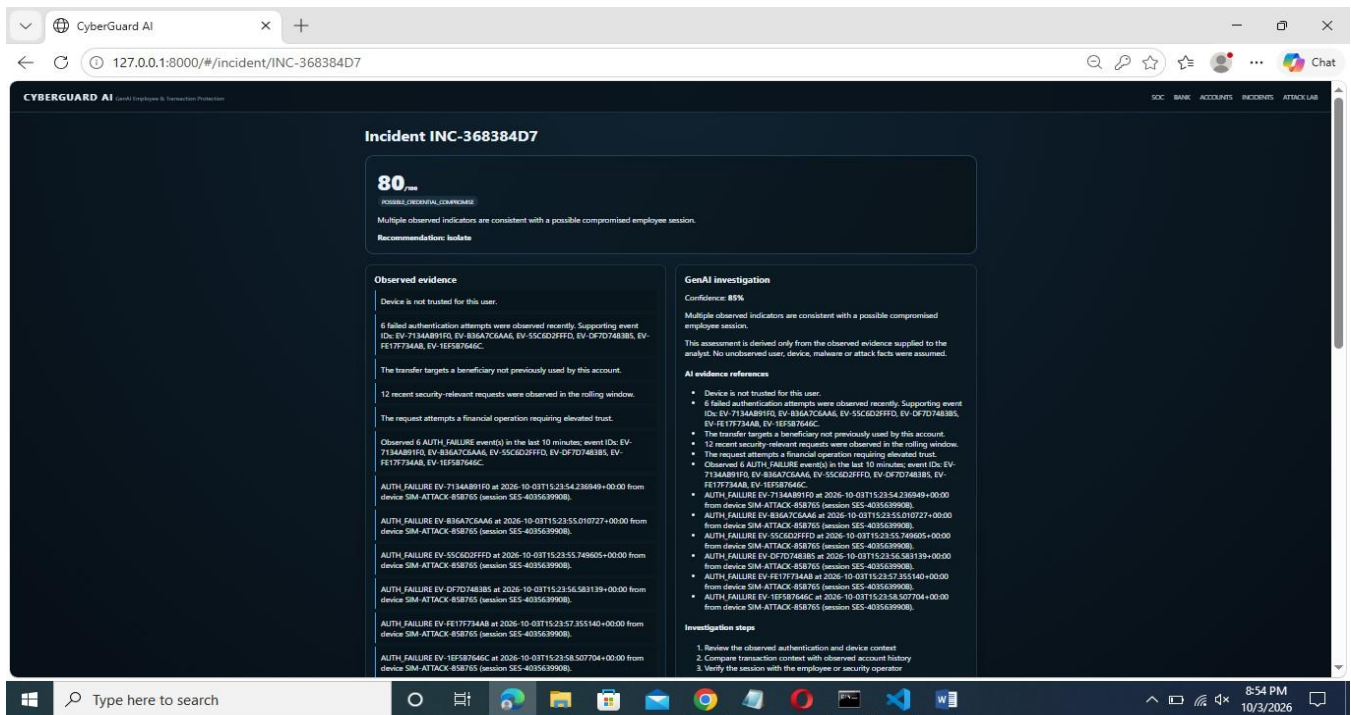


Fig: 8.6 Incident Investigation – GenAI Security Report

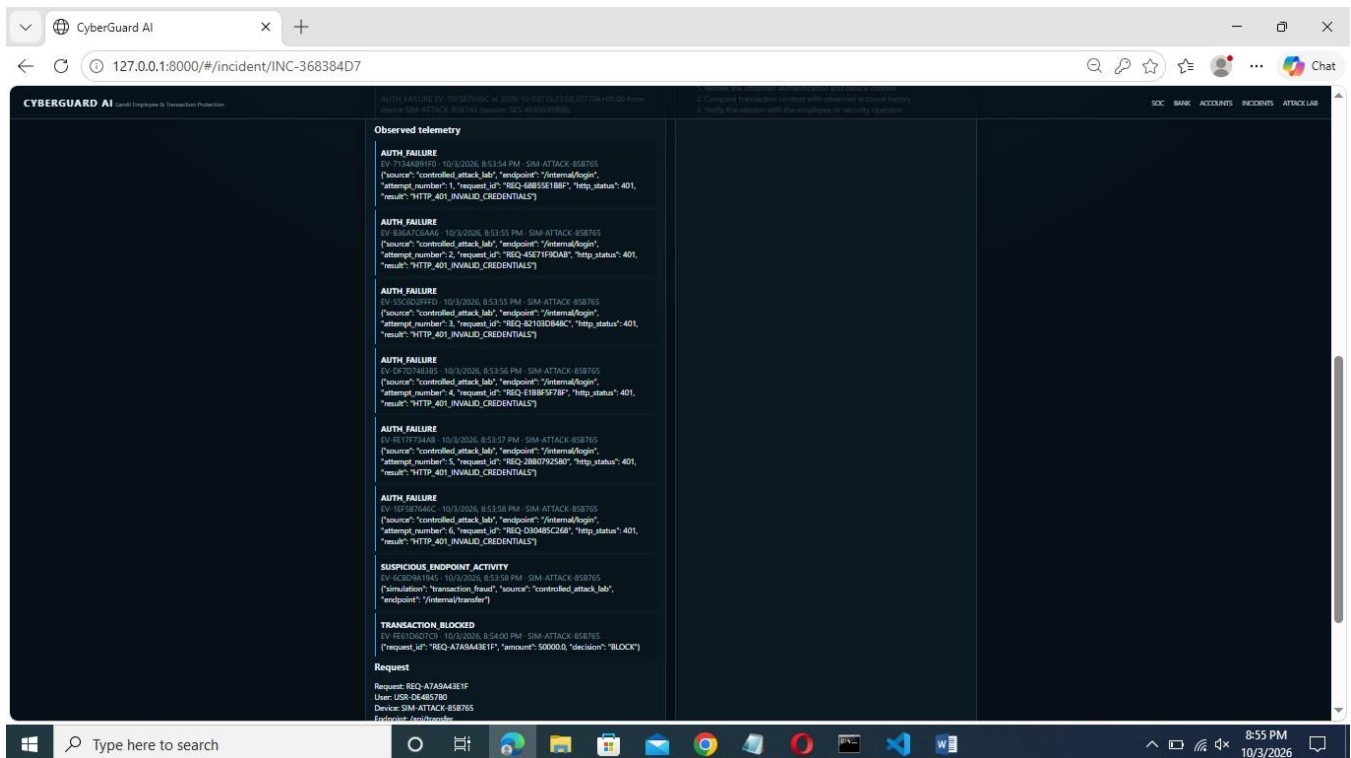


Fig: 8.7 Incident Investigation – Security Evidence & Telemetry

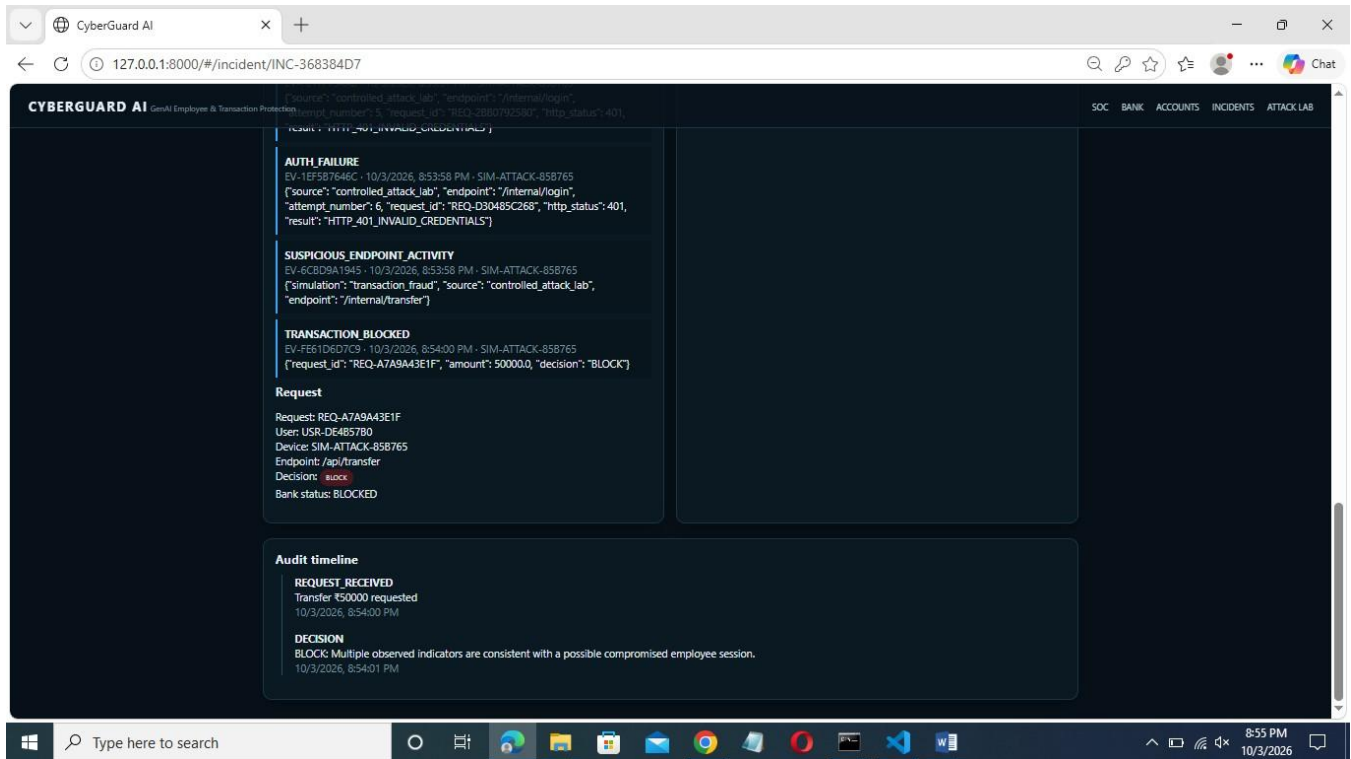


Fig:8.8 Incident Investigation – Audit Timeline

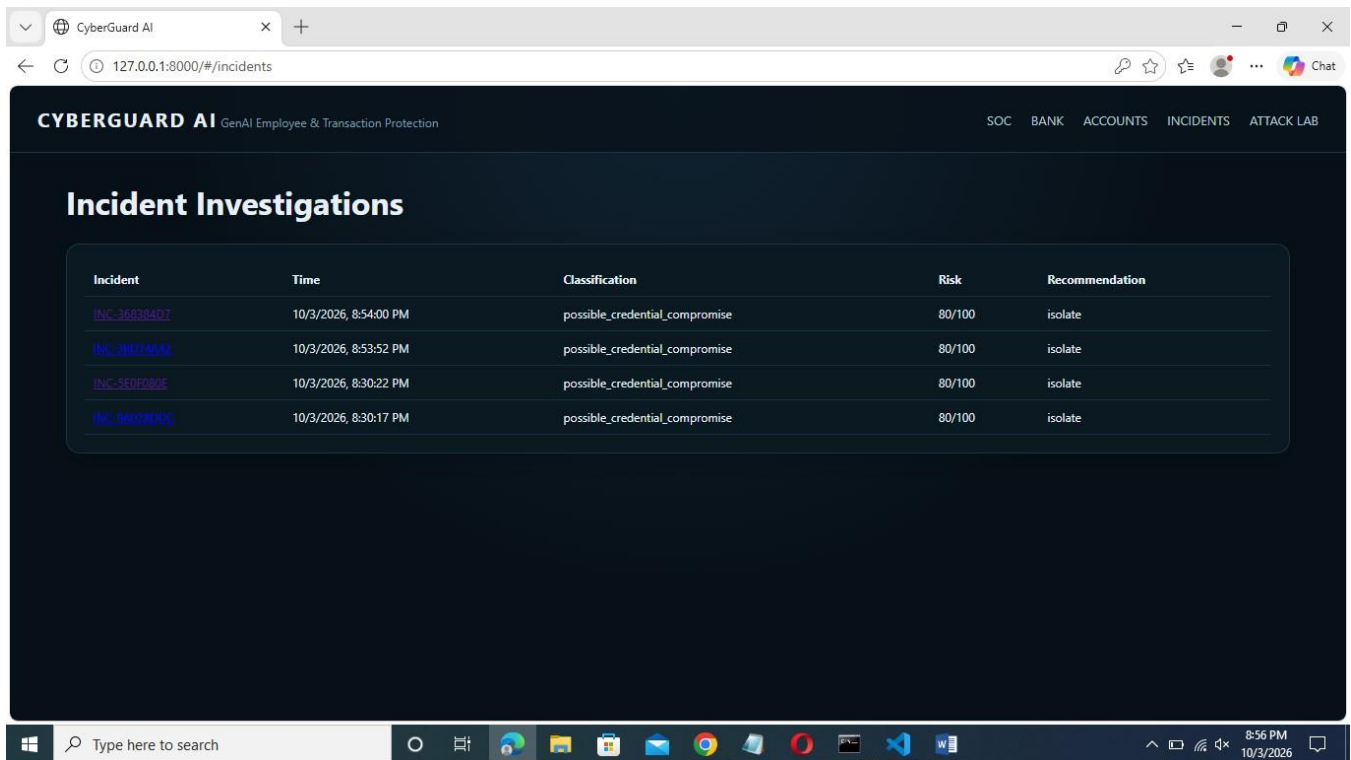


Fig: 8.9 Incident Investigations Dashboard

9. Challenges and Limitations

Challenges

- Maintaining a realistic banking workflow while keeping the entire environment synthetic and safe.
- Ensuring that security evidence is generated by actual requests rather than hard-coded incident data.
- Connecting deterministic security signals to a GenAI investigation without allowing the model to invent facts.
- Handling GenAI API failure safely through an evidence-backed fallback path.
- Managing two cooperating FastAPI services: the public CyberGuard Gateway and the protected synthetic Bank Server.
- Keeping database initialization, reset, account creation, and controlled attack execution reproducible for demonstrations.

Limitations

- The current application is a synthetic banking demonstration and should not be presented as a production banking system.
- GenAI quality depends on the configured model/API and network availability.
- SQLite is appropriate for the current demonstration but requires a persistent volume or migration to a production database for scalable deployment.
- The present interface is designed for a browser-based demonstration; mobile-native packaging is not part of the current implementation.
- Risk scoring is deterministic and explainable but uses a fixed set of indicators and weights rather than a trained fraud-detection model.
- Production deployment would require additional controls such as TLS termination, stronger secret management, identity integration, monitoring, rate limiting, database hardening, and security testing.

10. Conclusion

CyberGuard AI implements a functional GenAI cybersecurity application around a synthetic employee banking environment. The project goes beyond a static dashboard by connecting real user-created accounts, real demo balances, real transfer operations, controlled security telemetry, deterministic risk assessment, GenAI investigation, incident reporting, and enforcement.

The central GenAI contribution is the evidence-grounded security analyst. GenAI receives structured observations and deterministic risk context, produces a structured investigation, references concrete evidence where available, communicates confidence and uncertainty, and recommends an action. The application also records a deterministic fallback when GenAI is unavailable, preserving safe operation and traceability.

The resulting workflow demonstrates a practical security principle: detection should not be separated from enforcement. CyberGuard observes activity, investigates it, and applies the decision before the protected synthetic bank commits a suspicious operation. This creates a clear live demonstration of Detect → Investigate → Verify Evidence → Decide → Protect.

11. Future Scope

- Deploy the gateway and protected bank as cloud services with HTTPS and managed secrets.
- Migrate SQLite to PostgreSQL with persistent storage and transactional controls.
- Integrate enterprise identity providers and device-attestation signals.
- Add richer behavioural baselines and adaptive risk thresholds.
- Introduce multimodal GenAI analysis for security screenshots, log files, and structured reports where appropriate.
- Add human-in-the-loop analyst approval for medium-confidence or uncertain incidents.
- Add notification channels for high-severity incidents.
- Add role-based access control and multi-user SOC workflows.
- Add stronger evaluation metrics for GenAI grounding, hallucination rate, investigation accuracy, latency and cost.
- Scale telemetry collection using a message queue and centralized observability stack.
- Perform formal security testing, threat modelling, dependency scanning, API authentication hardening and penetration testing before any real-world use.

12. References

- CyberGuard AI project source code and README (v4 Evidence Fixed build).
- CyberGuard AI DEPLOYMENT.md — deployment configuration and service topology.
- CyberGuard AI gateway/ai.py — Gemini-compatible GenAI integration and evidence-grounded fallback analyst.
- CyberGuard AI gateway/core.py — evidence collection and incident investigation workflow.
- CyberGuard AI gateway/risk.py — deterministic risk indicators and 0–100 risk scoring.
- FastAPI documentation — backend API framework used by the application.
- Uvicorn documentation — ASGI server used to run the FastAPI services.
- HTTPX documentation — HTTP client used for service and GenAI API communication.
- Google Gemini API documentation — configured Generative AI integration endpoint.
- Python sqlite3 / SQLite documentation — demonstration persistence layer.